Cetus - DCA

Security Assessment

CertiK Assessed on Nov 26th, 2025





CertiK Assessed on Nov 26th, 2025

Cetus - DCA

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Exchange	Sui (SUI)	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE
Move	Preliminary comments published on 10/17/2025
	Final report published on 11/26/2025

Vulnerability Summary



14	11	0	3	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
0	Medium		Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
6	Minor	5 Resolved, 1 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
7	Informational	6 Resolved, 1 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | CETUS - DCA

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

External Dependencies

Privileged Roles

Upgradeability

Findings

CED-06 : Centralization Related Risks And Upgradability

CED-04 : Concerns About The Current Signature Message And Verification

CED-07 : Missing Zero Validations

CED-09 : `remove_role()` Does Not Check Whether The Role Exists

CED-11 : Out Amount Variable Lack Validation

CED-12 : When The Order'S InCoin Or OutCoin Is Not In The Whitelist, Making A Deal Should Be Disallowed

CED-13 : Lack Of Validation For `fee_rate`

CED-01 : Typos

CED-03 : Front-Running Issue In `flash_loan_for_make_deal()`

CED-05 : Discussion On Fee Included `in_amount`

CED-15 : Missing Validation On The Input `whitelist_mode`

CED-16 : Missing Check On The Length Of `signature`

CED-17 : Local Variable Naming

CED-18 : Function Lacks Version Control

Appendix

Disclaimer

CODEBASE | CETUS - DCA

Repository

https://github.com/CetusProtocol/cetus-clmm-sui/tree/482dc4009a3e95a117ae59752c3249f46d54e4cf

https://github.com/CetusProtocol/cetus-clmm-sui/tree/c6b5cb9c6304525f5673f8f748ec254b96b255a0

https://github.com/CetusProtocol/cetus-clmm-sui/tree/680dca8ae3f91a63078b56f1f6880b87daafe264

Commit

482dc4009a3e95a117ae59752c3249f46d54e4cf

c6b5cb9c6304525f5673f8f748ec254b96b255a0

680dca8ae3f91a63078b56f1f6880b87daafe264

788e10370cac04ce059875de9acd55e934ce441f

AUDIT SCOPE | CETUS - DCA

CetusProtocol/cetus-clmm-sui
cetus_dca/sources/config.move
cetus_dca/sources/acl.move
cetus_dca/sources/order.move
cetus_dca/sources/acl.move
cetus_dca/sources/config.move
cetus_dca/sources/order.move
cetus_dca/sources/acl.move
cetus_dca/sources/config.move
cetus_dca/sources/order.move

APPROACH & METHODS | CETUS - DCA

This audit was conducted for DCA to evaluate the security and correctness of the smart contracts associated with the Cetus - DCA project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | CETUS - DCA

Overview

The **Cetus - DCA** protocol is built on the Sui blockchain. This audit focuses on the following modules:

- `acl` : Implements bitmask-based roles over a `LinkedTable` , providing functions for setting, adding, removing, and managing members for authorization control.
- `config` : Defines the central policy and state (`GlobalConfig`), including version checks, timing and threshold parameters, per-direction coin whitelists.
- `order` : Manages the DCA order lifecycle and indexer. It enforces oracle and keeper checks, maintains whitelists, and handles operations such as `flash_loan_for_make_deal` , which splits input per cycle and issues a linear receipt, and `repay_for_make_deal` , which validates outputs, records execution, and closes orders upon completion.

External Dependencies

The project is developed using the Move language and running on the top of the Sui blockchain. The vulnerability and the updates of the language/Sui framework may affect the project as a whole. As the Sui network is rapidly evolving, to avoid any potential compatibility issues and take advantage of new features and improvements, the client should upgrade the Sui framework to the most recent version. Additionally, staying informed about any upcoming updates or changes to the language or framework can help ensure the project remains secure and compatible.

Dependency of the **Cetus - DCA**:

```
MoveSTL = { git = "https://github.com/CetusProtocol/move-stl.git", rev = "mainnet-v1.3.0", override = true }
```

Also, the project relies on off-chain keeper heavily, which needs to be carefully managed.

The above dependencies are not within the current audit scope and serve as a black box. Modules/Contracts within the module are assumed to be valid and non-vulnerable actors in this audit and implement proper logic to collaborate with the current project and other modules.

Privileged Roles

To set up the project correctly and ensure that the project functions properly, owners of the following objects are able to use privileged functions, more details in **CED-06: Centralization Related Risks And Upgradability**.

The advantage of the privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to serve the community best. It is also worthy of note the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the key pairs of privileged accounts are compromised, the project could have devastating consequences.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Furthermore, any plan to invoke the aforementioned functions should also be considered to move to the execution queue of

the `Timelock` contract.

Upgradeability

Developers working with the Sui blockchain have the ability to upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and publisher's key store should be handled with caution to prevent any unexpected loss. Additionally, it is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

Reference:

- Sui Package Upgrades
- Custom Policies

FINDINGS | CETUS - DCA



14	0	1	0	0	6	7
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for DCA to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 14 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
CED-06	**Centralization Related Risks And Upgradability**	**Centralization**	**Centralization**	**Acknowledged**
CED-04	Concerns About The Current Signature Message And Verification	Inconsistency	Minor	Resolved
CED-07	Missing Zero Validations	Volatile Code	Minor	Resolved
CED-09	`remove_role()` Does Not Check Whether The Role Exists	Volatile Code	Minor	Resolved
CED-11	Out Amount Variable Lack Validation	Logical Issue	Minor	Resolved
CED-12	When The Order'S InCoin Or OutCoin Is Not In The Whitelist, Making A Deal Should Be Disallowed	Logical Issue	Minor	Acknowledged
CED-13	Lack Of Validation For `fee_rate`	Logical Issue	Minor	Resolved
CED-01	Typos	Coding Issue	Informational	Resolved
CED-03	Front-Running Issue In `flash_loan_for_make_deal()`	Logical Issue	Informational	Resolved
CED-05	Discussion On Fee Included `in_amount`	Coding Style	Informational	Resolved

ID	Title	Category	Severity	Status
CED-15	Missing Validation On The Input `whitelist_mode`	Coding Issue	Informational	Resolved
CED-16	Missing Check On The Length Of `signature`	Coding Style	Informational	Resolved
CED-17	Local Variable Naming	Volatile Code	Informational	Resolved
CED-18	Function Lacks Version Control	Logical Issue	Informational	Acknowledged

CED-06 | Centralization Related Risks And Upgradability

Category	Severity	Location	Status
Centralization	**Centralization**		**Acknowledged**

Description

The role **ACL_MANAGER** has authority over the functions:

- config::set_whitelist_mode()
- config::set_min_cycle_frequency()
- config::set_oracle_valid_duration()
- config::add_in_coin_type()
- config::remove_in_coin_type()
- config::add_out_coin_type()
- config::remove_out_coin_type()
- config::set_keeper_threshold()
- config::claim_fee()

The role **ACL_CLAIM_FEE** and **OperatorCap** has authority over the functions:

- config::claim_fee_with_cap()

The role **AdminCap** has authority over the functions:

- config::mint_operator_cap()
- config::update_package_version()
- config::set_roles()
- config::remove_member()

The following functions require data signed by the **ACL_KEEPER** role to be called:

- order::flash_loan_for_make_deal_v2() and only keeper can invoke `flash_loan_for_make_deal_v2`.

The following functions require data signed by the **ACL_ORACLE** role to be called:

- order::open_order()

In addition, developers working with the Sui blockchain can upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and deployer's key store should be handled carefully to prevent any unexpected losses. It is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

More information can be found:

- Sui Package Upgrades
- Third-Party Package upgrades

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR

- Remove the risky functionality.

Alleviation

[Cetus, 11/05/2025]: These known functions are necessary for configuring, user identity recognition, and role permission control management when implementing DCA functionality, and they are necessary.

In addition, we have adopted multiple signatures when managing contracts to prevent the leakage of individual user permissions.

[CertiK, 11/05/2025]: It is suggested to implement the aforementioned methods to avoid centralized failure. The finding will be updated when further information is provided.

CED-04 Concerns About The Current Signature Message And Verification

Category	Severity	Location	Status
Inconsistency	Minor	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 452~458, 475~485	Resolved

Description

Order

In `validate_order`, the signed message omits parameters including the **`OutCoin`** type, **`min_out_amount_per_cycle`**, **`max_out_amount_per_cycle`**, and any **unique identifier or nonce**.

As a result, a valid oracle signature can be replayed by the same sender within the permitted timestamp window to create multiple orders. Because the omitted fields are not bound by the attestation, the same signature can also be reused to open orders with different OutCoin selections or arbitrary per-cycle min/max constraints, so long as the remaining inputs and timestamp align.

Deal

In `validate_keepers`, signatures are not bound to a specific executor address and do not include a per-order or per-cycle nonce.

This allows any address to consume a valid keeper signature and enables same-cycle replay or front-running, since there is no one-time value baked into the message. While cross-cycle replay is generally curtailed by state changes (e.g., next_cycle_at and amount_left_next_cycle), replay within a single cycle remains viable, particularly when cycle_frequency is set to zero.

Others

The signature payloads are not properly bound to their execution environment. Specifically, they lack essential contextual identifiers such as the chain ID, package/module ID, and protocol version. Moreover, they do not enforce an explicit expiration or deadline, even though the data payload includes time-related fields like `order.next_cycle_at` and `timestamp`:

```
data.append(bcs::to_bytes(&timestamp));
...
data.append(bcs::to_bytes(&order.next_cycle_at));
```

In addition, the signed `data` is constructed without domain separation or an intent prefix, contrary to Sui's recommended intent-signing approach. Without these bindings, valid signatures risk unintended reuse across modules, deployments, or future protocol versions (including after upgrades or hard forks). To mitigate replayability and scope ambiguity, bind all order-

and keeper-critical fields as well as the caller, environment identifiers (chain ID, package ID/module, protocol version), and a strict deadline, and adopt domain separation or intent signing as described in the Sui documentation: Sui intent signing.

Recommendation

Recommend revisiting the design of the signature message and adding necessary parts, for example:

- Include every parameter actually used during order creation (e.g., the chosen asset/type and any per-cycle limits)
- Bind deal signature with specific executor(depends on design)
- Add a unique id/nonce to ensure the signature can only be used once
- Incorporate domain separation and environment identifiers—chain ID, package ID, module/function, and a version field—to prevent cross-module or cross-deployment reuse
- Deadline(depends on design)

Alleviation

[Cetus, 11/05/2025]: Issue acknowledged. Changes have been reflected in the commit hash: https://github.com/CetusProtocol/cetus-clmm-sui/pull/46/commits/c6b5cb9c6304525f5673f8f748ec254b96b255a0

We added executable permission verification (handled in CED03) to the flash_loan_for_make_deal function to ensure it can only be executed by the keeper. Additionally, we introduced validate_keepers_v2 to enhance validity checks, ensuring the identity and timing of keepers when executing orders. This prevents order signature forgery and replay attacks.

[CertiK, 11/06/2025]:

Order

It is still possible to use the same signature multiple times within the valid time window to create multiple orders.

Additionally, since `OutCoin` **type**, `min_out_amount_per_cycle`, and `max_out_amount_per_cycle` are not included in the signed fields, the created orders may not align with the original intent when the signature was issued.

To address the signature replay issue, a unique signature identifier is needed — once a signature is used, it should be marked as consumed and become invalid for further use. For the three fields（`OutCoin` **type**, `min_out_amount_per_cycle`, and `max_out_amount_per_cycle`） not included in the signature, it is recommended to add them to the signed data to ensure the user's actions match the original signing intent.

Deal && Others

Currently, `flash_loan_for_make_deal` is restricted to be initiated by the keeper and checks the expiration time `timestamp + config.oracle_valid_duration()`, which resolves both the front-running and outdated signature issues. However, we still recommend adding a one-time signature verification, as suggested for the order, to prevent potential risks.

In addition, if identical contracts exist on the same or different chains and use the same set (or at least the minimum number required by `config.keeper_threshold()`) of keepers, the signatures may be subject to double-spending.We recommend adding a chain identifier as well as package and version identifiers to prevent such scenarios.

[Cetus, 11/07/2025]:

min_out_amount_per_cycle **and **max_out_amount_per_cycle are the upper and lower limits of the target token assets that users expect to receive when creating DCA orders.

The value used by the keeper when creating transactions and executing them is **promise_out_omount**.

In addition, the keeper synchronizes and verifies the validity of orders on the chain during signature execution, ensuring that the keeper does not generate duplicate orders in the same cycle (after order execution is completed, it enters the **next_cycle_at**).

A sub-transaction of a DCA order will only be executed by one keeper

---

[CertiK, 11/07/2025]:

However, when calling `open_order()` to create an order, the execution is not performed by the keeper — instead, the signature is sent to the user, who executes it themselves. As a result, the user can use the same signature multiple times before it expires, meaning there is no guarantee that a single signature can only be used once.

According to your explanation, `min_out_amount_per_cycle` and `max_out_amount_per_cycle` are user-defined parameters, and thus are not included in the signature scope. However, in `validate_order()`, the `OutCoin` is also not included in the signed data:

`order.move`

```
490     let mut data = type_name::get<InCoin>().into_string().into_bytes();
491     data.append(sender.to_bytes());
492     data.append(bcs::to_bytes(&cycle_frequency));
493     data.append(bcs::to_bytes(&cycle_count));
494     data.append(bcs::to_bytes(&fee_rate));
495     data.append(bcs::to_bytes(&in_amount_limit_per_cycle));
496     data.append(bcs::to_bytes(&timestamp));
497 
498     let (passed, _) = verify_internal(config, data, &signature, CHECK_ORACLE);
499     assert!(passed, EInvalidOracleSignature);
```

This allows the user to arbitrarily modify the `OutCoin` passed in, which may conflict with the original intent at the time of signing.

In addition, if identical contracts exist on the same or different chains and use the same set (or at least the minimum number required by `config.keeper_threshold()`) of keepers, the signatures may be subject to double-spending across different contracts with the same code. We recommend adding a chain identifier as well as package and version identifiers to prevent such scenarios.

[Cetus, 11/12/2025]:

The validate_order() function is private and cannot be called externally. It is only used within the open_order() function in the order.move module, which includes version checks for software packages.

For the open_order() business scenario in DCA, users utilize their existing in_coin_type to obtain the desired out_coin_type. Therefore, when creating an order, users must confirm the input currency type and the order execution period. The output

currency type is set by the user and represents the token they wish to exchange.

[CertiK, 11/12/2025]:

In open_order, we want to address the same signature can be reused multiple times:

A single oracle signature can be replayed to open multiple orders, including with different OutCoin types and different min_out_amount_per_cycle/max_out_amount_per_cycle, because those fields are not included in the signed payload, and there is no nonce or unique identifier to prevent reuse.

[Cetus, 11/13/2025]:

The function **open_order()** is used to create DCA orders, and the intention of using signatures here is to prevent the creation of malicious orders (splitting a small amount_in of order into multiple executions, consuming the gas fee of the keeper robot).Therefore, when creating DCA orders, it is necessary to verify the **input_comin/cycle_comount/in_amount_per_cycle **(the minimum amount of a single split order), and output relevant parameters without verification.

In addition, DCA orders created by replaying the same signature will not have an impact on the service. These orders will be considered as normal order creations, and a unique ID (Object ID) will be generated and saved in the **OrderIndexer** after creation. Subsequent replays cannot modify the parameters related to order output through** add()**.

[CertiK, 11/18/2025]:

As a follow-up question, the current implementation does not check if the inCoin and outCoin types are the same when creating an order.

We would like to check with the team if such validation has been performed offline or has the scenario has been considered.

[Cetus, 11/18/2025]: Fixed in the commit 788e10370cac04ce059875de9acd55e934ce441f by avoiding similar in and out coin types.

[CertiK, 11/18/2025]:

The finding is marked as resolved since the current implementation is intended and necessary validations has been added.

CED-07 | Missing Zero Validations

Category	Severity	Location	Status
Volatile Code	Minor	cetus_dca/sources/config.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 325	Resolved

Description

The configuration function `set_keeper_threshold` permits setting the keeper approval threshold to 0, and the corresponding validation logic treats `threshold = 0` as satisfied with an empty signature set. In this state, keeper-gated flows guarded by `validate_keepers` no longer require any keeper signatures, allowing any address to execute actions that were intended to require multi-party authorization. This effectively downgrades a multi-signature control to a no-signature control.

Similarily, the parameter `min_cycle_frequency` appears to lack a strict lower bound or is allowed to be set to zero. When `min_cycle_frequency = 0`, cycle executions can occur back-to-back within the same block or timestamp window, undermining rate-limiting assumptions and enabling same-cycle replay or rapid repeated execution. This directly interacts with the replay concerns noted in CED-04.

Recommendation

Recommend adding zero validation for the parameters mentioned above.

Alleviation

[Cetus, 11/07/2025]: The team heeded the advice and resolved the issue in commit 680dca8ae3f91a63078b56f1f6880b87daafe264

CED-09 | `remove_role()` Does Not Check Whether The Role Exists

Category	Severity	Location	Status
Volatile Code	Minor	cetus_dca/sources/acl.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 57~63	Resolved

Description

The `remove_role()` function does not check whether the specified role is already assigned to the member. If the role is absent, the subtraction operation (*perms - (1 << role)) may trigger an abort or inadvertently modify other role configurations.

`acl.move`

```
57      public fun remove_role(acl: &mut ACL, member: address, role: u8) {
58          assert!(role < 128, ERoleNumberTooLarge);
59          if (acl.permissions.contains(member)) {
60              let perms = acl.permissions.borrow_mut(member);
61              *perms = *perms - (1 << role);
62          }
63      }
```

Proof of Concept

The following test is trying to demonstrate the issue:

```
    #[test]
    fun test_remove_role_on_unset_bit_corrupts_other_bits() {
        let mut ctx = tx_context::dummy();
        let mut acl = TestACL {
            id: object::new(&mut ctx),
            acl: cetus_dca::acl::new(&mut ctx),
        };

        add_role(&mut acl.acl, @0xABCD, 5);
        assert!(has_role(&acl.acl, @0xABCD, 5), 0);
        assert!(!has_role(&acl.acl, @0xABCD, 4), 1);

        // Remove an UNSET role. With the current implementation this corrupts bits:
        // (1 << 5) - (1 << 4) => clears role 5 and sets role 4.
        remove_role(&mut acl.acl, @0xABCD, 4);

        // These assertions demonstrate the buggy effect and will PASS until the bug
is fixed.
        assert!(!has_role(&acl.acl, @0xABCD, 5), 2);
        assert!(has_role(&acl.acl, @0xABCD, 4), 3);

        transfer::transfer(acl, tx_context::sender(&ctx));
    }
```

Recommendation

It is recommended to adopt code similar to that in the `acl.move` of cetus-dlmm or check the role first:

```
public fun remove_role(acl: &mut ACL, member: address, role: u8) {
    assert!(role < 128, ERoleNumberTooLarge);
    if (acl.permissions.contains(member)) {
        let perms = acl.permissions.borrow_mut(member);
        *perms = *perms & (MAX_U128 - (1 << role));
    } else {
        abort EMemberNotExists
    }
}
```

Alleviation

[Cetus, 11/07/2025]: The team heeded the advice and resolved the issue in commit [680dca8ae3f91a63078b56f1f6880b87daafe264]

CED-11 | Out Amount Variable Lack Validation

Category	Severity	Location	Status
Logical Issue	Minor	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 101~102, 161, 246	Resolved

Description

`min_out_amount_per_cycle` and `max_out_amount_per_cycle` should be validated during `open_order()` to ensure that `min_out_amount_per_cycle <= max_out_amount_per_cycle`.

Also, `promise_out_amount` should be validated during `flash_loan_for_make_deal()` to ensure that `promise_out_amount >= min_out_amount_per_cycle && promise_out_amount <= max_out_amount_per_cycle`.

Recommendation

It is recommended to add these validations as described above.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue by adding validations in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-12 When The Order'S InCoin Or OutCoin Is Not In The Whitelist, Making A Deal Should Be Disallowed

Category	Severity	Location	Status
Logical Issue	Minor	cetus_dca/sources/config.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 238, 278, 288, 298, 308	Acknowledged

Description

When the contract uses `set_whitelist_mode()`, `remove_in_coin_type()`, or `remove_out_coin_type()` to remove an `InCoin` or `OutCoin` from the whitelist, trading of those tokens through the contract should be prohibited.

However, `flash_loan_for_make_deal()` does not check whether the traded `InCoin` and `OutCoin` are in the whitelist. This allows tokens that are no longer whitelisted to still be traded.

Recommendation

We would like to ask whether this is the intended design. If not, a `config.is_coin_allow<InCoin, OutCoin>` check should be added in `flash_loan_for_make_deal()`.

Alleviation

[Cetus, 11/05/2025]: The team acknowledged the issue, Subsequent DCA orders are initiated and executed by trusted keepers, and there is no need to perform further checks on existing orders.

[CertiK, 11/05/2025]: As the open order does not limit the `OutCoin` with keeper's signature, the following offchain process with the keeper should be carefully handled when trying to complete the order.

[Cetus, 11/19/2025]: The initial design of the coin whitelist was set up to avoid generating orders that could not be executed and to alleviate the transaction pressure on the keeper robot, and is not related to fund security. At present, all types of coins have been opened without any restrictions.

CED-13 | Lack Of Validation For `fee_rate`

Category	Severity	Location	Status
Logical Issue	Minor	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 435	Resolved

Description

During `open_order()`, there is a lack of validation for the input `fee_rate`. The `fee_rate` should be less than a specific threshold (at least less than `FEE_RATE_DENOMINATOR`) to prevent a DOS caused by fees exceeding 100% and make_deal fees being so high that no one executes deals.

Recommendation

We recommend setting an upper limit for `fee_rate` and validating it during `open_order`.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue by adding validation on the fee rate in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0

CED-01 | Typos

Category	Severity	Location	Status
Coding Issue	Informational	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 84, 389~390	Resolved

Description

The emitted event type is misspelled ("Cycel"), which can cause indexers or alerts expecting the canonical spelling to miss the event.

The public function is misspelled as `cancle_order()`. This can confuse users and cause call failures in client scripts or tooling that expect the conventional spelling `cancel_order()`.

Recommendation

It is recommended to fix typos to avoid confusion.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-03 | Front-Running Issue In `flash_loan_for_make_deal()`

Category	Severity	Location	Status
Logical Issue	Informational	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 475~480	Resolved

Description

In the `flash_loan_for_make_deal()` function, the `validate_keepers()` process does not include `ctx.sender()` in the data used for signature verification. As a result, anyone holding the signature can execute the `flash_loan_for_make_deal()` transaction.

This means that anyone can front-run the transaction, especially when a potential arbitrage opportunity exists.

Recommendation

We would like to confirm whether the team has considered this issue and whether it could impact the DCA process.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue by adding validation on `ctx.sender()` in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-05 | Discussion On Fee Included `in_amount`

Category	Severity	Location	Status
Coding Style	Informational	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 294, 335	Resolved

Description

`MakeDealReceipt.in_amount` and `MakeDealEvent.in_amount` record the gross pre-fee input while execution returns net-of-fee coins.

We would like to check with the team if this is intended design.

Recommendation

We would like to check with the team if this is intended design.

Alleviation

[Cetus, 11/04/2025]: Issue acknowledged. I won't make any changes for the current version.

Reason: We aim to record the user's original in_amount in the 'MakeDealEvent' event. Additionally, the event separately logs the fee_amount, which is used to calculate the lending amount after deducting the transaction fee.

CED-15 | Missing Validation On The Input `whitelist_mode`

Category	Severity	Location	Status
Coding Issue	Informational	cetus_dca/sources/config.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 238~239	Resolved

Description

The `whitelist_mode` field in `GlobalConfig` determines how whitelist checks are applied:

- `0` : Whitelist disabled — all coins are allowed.
- `1` : Only `InCoin` is checked and must appear in `in_coin_whitelist` .
- `2` : Only `OutCoin` is checked and must appear in `out_coin_whitelist` .
- `3` : Both input and output coins must be whitelisted.

As a result, only the values `0` , `1` , `2` , and `3` should be used.

Recommendation

It is recommended to validate `whitelist_mode` to ensure the value is less than or equal to 3.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue by adding validation on the whitelist_mode in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-16 | Missing Check On The Length Of `signature`

Category	Severity	Location	Status
Coding Style	Informational	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 499~500, 503~504	Resolved

Description

The `parse_signature()` function divides the input `signature` into two components: the `sign` (actual signature bytes) and the `public_key` (used to compute the corresponding `address`).

```
let mut public_key = vector<u8>[0];
let mut sign = vector[];
let (mut i, length) = (1, vector::length(&signature));
let signature_length = length - 32 - 1;
while (i < length) {
    if (i <= signature_length) {
        sign.push_back(*vector::borrow(&signature, i));
    } else {
        public_key.push_back(*vector::borrow(&signature, i));
    };
    i = i + 1;
};
let address = address::from_bytes(hash::blake2b256(&public_key));
```

In this implementation, the **last 32 bytes** of the `signature` represent the public key, which is hashed using `blake2b256` to derive the `address`.

The extracted `sign` portion corresponds to the actual signature used in `ed25519::ed25519_verify()`. According to the official Sui documentation, an Ed25519 signature must be exactly **64 bytes** in length.

As a result, the complete `signature` must consist of **97 bytes** in total:

- 1 byte for the `flag`,
- 64 bytes for the `sign`, and
- 32 bytes for the `public_key`.

If the input `signature` does not match this expected length, the function will abort due to invalid data size.

Reference: Sui Documentation – Signatures

Recommendation

It is recommended to add a length check for the input signature before processing to prevent unexpected aborts or out-of-

bounds access.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue by adding validation on signature length in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-17 | Local Variable Naming

Category	Severity	Location	Status
Volatile Code	Informational	cetus_dca/sources/order.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 527~528	Resolved

Description

A local variable is named address, the same as the imported module (sui::address). While this does not cause semantic shadowing for qualified calls (address::...), the name collision creates ambiguity.

- Users may misinterpret usages as shadowing, increasing cognitive load during audits, debugging, and integrations.
- Future refactors that introduce/modify a use alias (e.g., alias sui::address to address) or adjust imports can trigger name conflicts, compilation errors, or broken references.

Recommendation

It is recommended to use a different variable name instead of `address`.

Alleviation

[Cetus, 11/05/2025]: The team heeded the advice and resolved the issue in commit c6b5cb9c6304525f5673f8f748ec254b96b255a0.

CED-18 | Function Lacks Version Control

Category	Severity	Location	Status
Logical Issue	Informational	cetus_dca/sources/config.move (482dc4009a3e95a117ae59752c3249f46d54e4cf): 42	Acknowledged

Description

The following functions lack version control:

config

- mint_operator_cap()
- set_whitelist_mode()
- set_min_cycle_frequency()
- set_oracle_valid_duration()
- add_in_coin_type()
- remove_in_coin_type()
- add_out_coin_type()
- remove_out_coin_type()
- set_keeper_threshold()
- receive_fee()

In particular, `receive_fee()` can be called by anyone.

If these functions are not restricted by the corresponding version control, they will remain callable even after the contract upgrade.

Recommendation

We would like to verify that this behavior is by design. If it isn't, add explicit protocol/versioning guards (e.g., module/package ID, protocol version) to ensure compatibility and prevent misuse.

Alleviation

[Cetus, 11/07/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

APPENDIX | CETUS - DCA

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Web3 Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.